

Báo cáo Phân tích Kỹ thuật: Cơ chế Xác thực Bản quyền & Kỹ thuật Phòng vệ

Chuyên đề: Phân tích mã dịch ngược (Decompiled Code Analysis) & Kỹ thuật Mật mã học trong Reverse Engineering

Tóm tắt nội dung: Tài liệu phân tích toàn diện kiến trúc bảo mật của đoạn mã kiểm tra License phần mềm (Sublime Text / Sublime Merge). Hệ thống kết hợp cơ chế **Mật mã bất đối xứng RSA-1024** với **Kiểm tra toàn vẹn nhiều tầng (Defense in Depth)**, bao gồm làm rối mã (Obfuscation), đối chiếu mã băm (SHA-256 Checksum), bộ lọc danh sách đen (Blacklisting) và bẫy lỗi trễ (Delayed Probabilistic Failure).

1. Tổng quan Kiến trúc & Luồng Thực thi

Input License (Chuỗi ký tự do người dùng nhập)

↓

[sub_7FF7A50B2C25] → Parse cấu trúc (Tách dòng: Name, Type, Prefix-ID, Signature)

↓

→ Kiểm tra Prefix ("EA7E", "E3D2", cấm "E52D")

→ Kiểm tra ID qua 3 tầng Blacklist (Bitmask, Switch-case, Binary Search)

↓

[XOR 0xB3 Deobfuscation] → Giải mã RSA-1024 Public Key từ mảng byte tĩnh (.rdata)

↓

→ [SHA-256 Checksum] → Đối chiếu 4 byte kiểm tra tính toàn vẹn

↓

[sub_7FF7A50B2FC7] → Chuẩn hóa Payload ("Name
Type
Prefix-ID")

↓

[sub_7FF7A50B3613] → Xác thực Chữ ký số: RSA PKCS#1 v1.5 + SHA-1 (LibTomCrypt)

↓

[Delayed Failure Check] → Kiểm tra byte băm thứ 31 + Bẫy ngẫu nhiên (std::_Random_device)

2. Phân tích Chi tiết Từng Tầng Logic

Tầng 1: Giải mã Khóa công khai (Key Deobfuscation)

Tại hàm sub_7FF7A4CA951C, chương trình thực hiện bước giải mã khóa tĩnh:

```
for ( i = 0; i != 160; ++i )
{
    v10 = byte_7FF7A519D3E0[i] ^ 0xB3;
    *((_BYTE *)Src + 2 * i) = a0123456789abcd[(byte_7FF7A519D3E0[i] ^ 0xB3u) >> 4];
    *((_BYTE *)Src + 2 * i + 1) = a0123456789abcd[v10 & 0xF];
}
LOBYTE(Src[40]) = 0;
p_Dst = &Dst;
sub_7FF7A4C2E5E8(&Dst, (char *)Src);
```

- Cơ chế kỹ thuật:** Mảng byte_7FF7A519D3E0 (160 bytes) được lưu tĩnh trong phân vùng .rdata. Mỗi byte được XOR với hằng số 0xB3 để tái tạo khóa **RSA Public Key (ASN.1 DER SubjectPublicKeyInfo)**. Sau đó, nó được chuyển đổi sang chuỗi Hexadecimal độ dài 320 ký tự lưu vào Dst.

- **Mục đích thiết kế:** Chống rà soát tĩnh (*Anti-Static Signature Scanning*). Khóa công khai RSA chuẩn định dạng ASN.1 DER luôn có chuỗi byte đặc trưng `30 81 9d 30 0d 06 09 2a 86 48...`. Việc XOR `0xB3` giúp xóa bỏ chữ ký nhị phân này khi phần mềm bị quét bởi các công cụ như `strings` hoặc PE analyzer.

Tầng 2: Kiểm tra Tính toàn vẹn của Khóa (Anti-Tampering)

Ngay sau khi giải mã Public Key ra chuỗi Hex, hàm kiểm tra tính toàn vẹn:

```
sub_7FF7A4D0F650(a1: v38, a2: p_Dst, a3: v30);
v17 = 2;
if ( (v38[0] ^ 0xC6 | v38[18] ^ 0xEA | v38[30] ^ 0x56) == 0 )
    v17 = v16;
```

- **Cơ chế kỹ thuật:** Hàm `sub_7FF7A4D0F650` tính mã băm **SHA-256** của chuỗi Hex Public Key (lưu kết quả vào mảng 32 byte `v38`). Phép so sánh bitwise:
`(v38[0] ^ 0xC6) | (v38[18] ^ 0xEA) | (v38[30] ^ 0x56) == 0`
Biểu thức chỉ bằng 0 khi cả 3 byte kiểm tra đều khớp: `v38[0] == 0xC6`, `v38[18] == 0xEA`, `v38[30] == 0x56`.
- **Mục đích thiết kế:** Ngăn chặn tấn công *Public Key Substitution*. Nếu người phân tích cố gắng ghi đè mảng byte bằng một Public Key tự sinh khác, mã băm SHA-256 sẽ sai lệch hoàn toàn, khiến chương trình gán mã lỗi `v17 = 2` (Invalid).

Tầng 3: Phân tích Cấu trúc License (License Parsing)

Hàm `sub_7FF7A50B2C25` bóc tách dữ liệu người dùng nhập vào:

Thứ tự dòng	Thành phần	Mô tả & Quy tắc xử lý
Dòng 1	Licensee Name	Tên người dùng hoặc tên tổ chức được cấp phép.
Dòng 2	License Type	Phân loại giấy phép: <code>Single User License</code> (1 user), <code>Unlimited User License</code> (0 user), hoặc <code><N> User License</code> .
Dòng 3	Prefix - ID	Định dạng <code><Prefix>-<ID></code> . Prefix bắt buộc là <code>EA7E</code> hoặc <code>E3D2</code> (từ chối <code>E52D</code>). ID là số nguyên.
Dòng 4 – 11	RSA Signature	Gồm 8 dòng mã Hex (mỗi dòng 32 ký tự, tổng 256 ký tự Hex = 128 bytes). Đây là chữ ký số PKCS#1 v1.5.

Tầng 4: Bộ lọc Danh sách Đen nhiều tầng (Multi-tier Blacklisting)

Trước khi đưa chữ ký số vào giải mã, giá trị số `ID` ở Dòng 3 phải vượt qua 3 bộ lọc thu hồi license:

1. **Bitmask Check:** Sử dụng lệnh `_bittest64` kiểm tra dải số liên tiếp thông qua các hằng số bitmask `0x20000000000000581` và `0x8000010000000001` với độ phức tạp $O(1)$.
2. **Switch-case Lookup:** Bảng đối chiếu trực tiếp hàng chục ID cụ thể đã bị phát hiện chia sẻ trái phép công khai (ví dụ: `0xC1A2A`, `0xC1AA2`, ...).
3. **Binary Search Check:** Hàm `sub_7FF7A4CAA007` thực thi tìm kiếm nhị phân trên mảng dữ liệu tĩnh đã sắp xếp `unk_7FF7A518CFD0` (chứa hàng trăm ID 4-byte).

Tầng 5: Xác thực Chữ ký Mật mã (Cryptographic Verification)

Hàm `sub_7FF7A50B3613` sử dụng thư viện mật mã học **LibTomCrypt**:

```
memmove(&qword_7FF7A53123F0, &off_7FF7A5205B20, 0x1A0u); // Khởi tạo LibTomMath
sub_7FF7A4C2D68E(&off_7FF7A5205968); // Đăng ký thuật toán SHA-1
v6 = sub_7FF7A4C277BC("sha1");
sub_7FF7A4C2CF48(v6, a1, v7, v14, &v11); // SHA-1(Payload)
sub_7FF7A4C1EE02(v16, v8, v13); // rsa_import (Public Key)
sub_7FF7A4C2B8C1(v15, v9, v14, v11, 1, v6, 0, &v12, v13); // rsa_verify_hash (PKCS#1 v1.5)
```

- **Dữ liệu ký (Payload):** Được hàm `sub_7FF7A50B2FC7` định dạng theo chuẩn:

```
Payload = Name + " " + License Type + " " + Prefix-ID
```

- **Quy trình xác thực:**

```
Payload Hash = SHA-1(Payload)
```

```
Verify(PublicKey, Signature, Payload Hash) == True
```

Tầng 6: Kỹ thuật Phòng thủ Lỗi trễ (Delayed Probabilistic Trap)

Ở phần cuối của hàm kiểm tra bản quyền:

```
if ( ((v38[31] != 0xEA) & (unsigned __int8)v18) == 1 )
    v17 = 2 * ((std::_Random_device() & 0xF) == 0);
```

Phân tích cơ chế bẫy:

- Byte thứ 31 của SHA-256 (`v38[31]`) được kiểm tra tách biệt tại cuối hàm thay vì kiểm tra cùng lúc ở Tầng 2.
- Nếu byte này không bằng `0xEA`, chương trình **không báo lỗi ngay lập tức**. Thay vào đó, nó gọi `std::_Random_device()` để lấy số ngẫu nhiên phần cứng. Chỉ với xác suất `1/16` (khi 4 bit cuối bằng 0), nó mới làm hỏng trạng thái bản quyền (`v17 = 2`).
- **Mục đích:** Kỹ thuật này làm sai lệch kết quả kiểm thử động (Dynamic Analysis). Người phân tích có thể thấy bản vá của mình hoạt động ổn định trong vài lần chạy đầu, nhưng sau đó phần mềm sẽ đột ngột báo lỗi ngẫu nhiên, gây khó khăn lớn trong việc xác định nguyên nhân gốc rễ.

3. Đánh giá Mật mã học & Bài học Thực tiễn

1. Bảo mật dựa trên tính bất đối xứng của RSA:

Thuật toán RSA đảm bảo rằng chỉ người sở hữu *Private Key* tương ứng mới có thể tạo ra chữ ký số hợp lệ cho bất kỳ Payload nào. Dù toàn bộ mã nguồn kiểm tra và Public Key (`n`, `e`) bị lộ, việc giải mã ngược để tìm Private Key d tương đương với bài toán phân tích thừa số nguyên tố của hợp số 1024-bit `n`, điều bất khả thi với năng lực điện toán hiện tại.

2. Điểm giới hạn của Client-side Verification:

Mặc dù hệ thống xác thực được thiết kế rất chặt chẽ (kết hợp chữ ký số chuẩn, obfuscation, kiểm tra toàn vẹn và bẫy ngẫu nhiên), điểm yếu cố hữu của mọi phần mềm offline là toàn bộ mã nhị phân nằm trên máy tính của người dùng. Mọi lệnh kiểm tra logic cuối cùng đều phải quy về các lệnh rẽ nhánh điều kiện (như `test eax, eax; jz`) trong CPU.